

Código Java — Paradigma Orientado a Objetos

Explicação didática linha a linha — oo/FolhaPagamento.java

[Folha de Pagamento](#)

[Paradigmas](#)

[Linha a linha](#)

Como ler este código

Java organiza o domínio em objetos/records, interfaces e classes de regra. A leitura ideal é observar responsabilidades: dados, regras, cálculo e resumo.

Regras de negócio usadas

- $\text{valor_hora} = \text{salário_bruto} / 180$
- $\text{extras} = \text{valor_hora} \times 1,5 \times \text{horas extras}$
- $\text{base_irrf} = \text{bruto} + \text{extras} - \text{INSS} - \text{dependentes} \times 189,59$
- $\text{líquido} = \text{bruto} + \text{extras} - \text{INSS} - \text{IRRF}$
- Os mesmos três funcionários são usados em todas as versões: Ana, Bruno e Carla.

Visão geral da solução

A implementação produz os mesmos resultados verificados nas quatro linguagens: Total da folha = 6564.90; maior líquido = Ana (3317.07); menor líquido = Carla (1245.83).

Explicação linha a linha

Linha	Código	Explicação didática
1	<code>import java.util.List;</code>	Importa List, coleção usada para guardar funcionários e resultados.
2	<code>import java.util.Comparator;</code>	Importa Comparator, usado para comparar salários líquidos.
3	<code>import java.util.Locale;</code>	Importa Locale para padronizar separador decimal no formato americano.
4		Linha em branco para separar conceitos e melhorar legibilidade.
5	<code>public class FolhaPagamento {</code>	Define a classe pública principal do programa.
6	<code> public static void main(String[] args) {</code>	Método de entrada executado pela JVM.

Linha	Código	Explicação didática
7	<code>Locale.setDefault(Locale.US);</code>	Define ponto como separador decimal, facilitando a saída com duas casas.
8	<code>List<Funcionario> funcionarios = List.of(</code>	Cria uma lista imutável de funcionários de entrada.
9	<code>new Funcionario("Ana", 4500.00, 2, 8),</code>	Instancia um funcionário com nome, salário bruto, dependentes e horas extras.
10	<code>new Funcionario("Bruno", 2200.00, 0, 0),</code>	Instancia um funcionário com nome, salário bruto, dependentes e horas extras.
11	<code>new Funcionario("Carla", 1300.00, 1, 4)</code>	Instancia um funcionário com nome, salário bruto, dependentes e horas extras.
12	<code>);</code>	Fecha o bloco ou chamada iniciada anteriormente.
13		Linha em branco para separar conceitos e melhorar legibilidade.
14	<code>CalculadoraFolha calculadora = new CalculadoraFolha(new RegraINSS(), new RegraIRRF());</code>	Cria a calculadora recebendo regras de INSS e IRRF por composição.
15	<code>List<ResultadoFolha> resultados = calculadora.calcular(funcionarios);</code>	Calcula todos os funcionários e obtém a lista de resultados.
16	<code>ResumoFolha resumo = ResumoFolha.de(resultados);</code>	Gera os agregados: total, maior e menor líquido.
17		Linha em branco para separar conceitos e melhorar legibilidade.
18	<code>System.out.println("Folha de pagamento - paradigma orientado a objetos (Java)");</code>	Imprime texto ou cada resultado no console.
19	<code>System.out.println("Politica: valores monetarios exibidos com 2 casas decimais; calculos internos usam double.\n");</code>	Imprime texto ou cada resultado no console.
20	<code>resultados.forEach(System.out::println);</code>	Linha de código Java que participa da organização orientada a objetos da solução.
21	<code>System.out.printf("\nTotal da folha: %.2f\n", resumo.totalFolha());</code>	Imprime valores formatados com duas casas decimais.

Linha	Código	Explicação didática
22	<pre> System.out.printf("Maior salario liquido: %s (%.2f)%n", resumo.maior().nome(), resumo.maior().liquido());</pre>	Imprime valores formatados com duas casas decimais.
23	<pre> System.out.printf("Menor salario liquido: %s (%.2f)%n", resumo.menor().nome(), resumo.menor().liquido());</pre>	Imprime valores formatados com duas casas decimais.
24	<pre> }</pre>	Fecha o bloco ou chamada iniciada anteriormente.
25	<pre>}</pre>	Fecha o bloco ou chamada iniciada anteriormente.
26		Linha em branco para separar conceitos e melhorar legibilidade.
27	<pre>record Funcionario(String nome, double salarioBruto, int dependentes, int horasExtras) {}</pre>	Record que representa os dados de entrada de forma imutável e concisa.
28		Linha em branco para separar conceitos e melhorar legibilidade.
29	<pre>record ResultadoFolha(String nome, double bruto, double extras, double inss, double irrf, double liquido) {</pre>	Record que representa todos os valores calculados para um funcionário.
30	<pre> @Override</pre>	Indica que o método seguinte sobrescreve comportamento herdado.
31	<pre> public String toString() {</pre>	Define como um ResultadoFolha será convertido em texto.
32	<pre> return String.format("%-8s Bruto: %8.2f Extras: %7.2f INSS: %7.2f IRRF: %7.2f Liquido: %8.2f",</pre>	Monta a linha de saída formatada com alinhamento e duas casas.
33	<pre> nome, bruto, extras, inss, irrf, liquido);</pre>	Linha de código Java que participa da organização orientada a objetos da solução.
34	<pre> }</pre>	Fecha o bloco ou chamada iniciada anteriormente.
35	<pre>}</pre>	Fecha o bloco ou chamada iniciada anteriormente.
36		Linha em branco para separar conceitos e melhorar legibilidade.

Linha	Código	Explicação didática
37	<code>record Faixa(double limite, double aliquota) {</code>	Record que representa uma faixa de imposto: limite e alíquota.
38	<code> boolean contem(double valor) {</code>	Método que verifica se um valor pertence à faixa.
39	<code> return valor <= limite;</code>	Retorna verdadeiro quando o valor cabe no limite da faixa.
40	<code> }</code>	Fecha o bloco ou chamada iniciada anteriormente.
41	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
42		Linha em branco para separar conceitos e melhorar legibilidade.
43	<code>interface RegraDesconto {</code>	Define contrato para qualquer regra de desconto.
44	<code> double calcular(double base);</code>	Assinatura do cálculo: recebe uma base e devolve o desconto.
45	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
46		Linha em branco para separar conceitos e melhorar legibilidade.
47	<code>class RegraPorFaixas implements RegraDesconto {</code>	Implementa uma regra genérica baseada em faixas.
48	<code> private final List<Faixa> faixas;</code>	Guarda as faixas como dependência interna imutável.
49		Linha em branco para separar conceitos e melhorar legibilidade.
50	<code> RegraPorFaixas(List<Faixa> faixas) {</code>	Construtor que recebe a lista de faixas da regra concreta.
51	<code> this.faixas = faixas;</code>	Armazena a lista recebida no atributo da classe.
52	<code> }</code>	Fecha o bloco ou chamada iniciada anteriormente.
53		Linha em branco para separar conceitos e melhorar legibilidade.

Linha	Código	Explicação didática
54	<code>@Override</code>	Indica que o método seguinte sobrescreve comportamento herdado.
55	<code>public double calcular(double base) {</code>	Assinatura do cálculo: recebe uma base e devolve o desconto.
56	<code>double aliquota = faixas.stream()</code>	Inicia um pipeline funcional sobre as faixas.
57	<code>.filter(faixa -> faixa.contem(base))</code>	Mantém apenas faixas que contêm a base de cálculo.
58	<code>.findFirst()</code>	Seleciona a primeira faixa compatível.
59	<code>.orElseThrow()</code>	Falha se nenhuma faixa for encontrada, evitando resultado silenciosamente errado.
60	<code>.aliquota();</code>	Extraí a alíquota da faixa encontrada.
61	<code>return base * aliquota;</code>	Calcula o valor do desconto multiplicando base pela alíquota.
62	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
63	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
64		Linha em branco para separar conceitos e melhorar legibilidade.
65	<code>class RegraINSS extends RegraPorFaixas {</code>	Especializa RegraPorFaixas com a tabela do INSS.
66	<code>RegraINSS() {</code>	Linha de código Java que participa da organização orientada a objetos da solução.
67	<code>super(List.of(</code>	Chama o construtor da classe mãe com a lista de faixas.
68	<code>new Faixa(1412.00, 0.075),</code>	Define uma faixa: limite e alíquota aplicada.
69	<code>new Faixa(2666.00, 0.090),</code>	Define uma faixa: limite e alíquota aplicada.
70	<code>new Faixa(4000.00, 0.120),</code>	Define uma faixa: limite e alíquota aplicada.
71	<code>new Faixa(Double.POSITIVE_INFINITY, 0.140)</code>	Define uma faixa: limite e alíquota aplicada.
72	<code>));</code>	Fecha o bloco ou chamada iniciada anteriormente.

Linha	Código	Explicação didática
73	}	Fecha o bloco ou chamada iniciada anteriormente.
74	}	Fecha o bloco ou chamada iniciada anteriormente.
75		Linha em branco para separar conceitos e melhorar legibilidade.
76	class RegraIRRF extends RegraPorFaixas {	Especializa RegraPorFaixas com a tabela do IRRF.
77	RegraIRRF() {	Linha de código Java que participa da organização orientada a objetos da solução.
78	super(List.of(	Chama o construtor da classe mãe com a lista de faixas.
79	new Faixa(2259.00, 0.000),	Define uma faixa: limite e alíquota aplicada.
80	new Faixa(2826.00, 0.075),	Define uma faixa: limite e alíquota aplicada.
81	new Faixa(3751.00, 0.150),	Define uma faixa: limite e alíquota aplicada.
82	new Faixa(4664.00, 0.225),	Define uma faixa: limite e alíquota aplicada.
83	new Faixa(Double.POSITIVE_INFINITY, 0.275)	Define uma faixa: limite e alíquota aplicada.
84	));	Fecha o bloco ou chamada iniciada anteriormente.
85	}	Fecha o bloco ou chamada iniciada anteriormente.
86	}	Fecha o bloco ou chamada iniciada anteriormente.
87		Linha em branco para separar conceitos e melhorar legibilidade.
88	class CalculadoraFolha {	Classe de serviço que coordena o cálculo completo da folha.
89	private static final double DEDUCAO_POR_DEPENDENTE = 189.59;	Constante da dedução por dependente usada no IRRF.
90	private final RegraDesconto regraINSS;	Dependência da regra de INSS, acessada pelo contrato RegraDesconto.

Linha	Código	Explicação didática
91	<code>private final RegraDesconto regraIRRF;</code>	Dependência da regra de IRRF, também acessada pela interface.
92		Linha em branco para separar conceitos e melhorar legibilidade.
93	<code>CalculadoraFolha(RegraDesconto regraINSS, RegraDesconto regraIRRF) {</code>	Construtor que injeta as regras de desconto.
94	<code>this.regraINSS = regraINSS;</code>	Linha de código Java que participa da organização orientada a objetos da solução.
95	<code>this.regraIRRF = regraIRRF;</code>	Linha de código Java que participa da organização orientada a objetos da solução.
96	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
97		Linha em branco para separar conceitos e melhorar legibilidade.
98	<code>List<ResultadoFolha> calcular(List<Funcionario> funcionarios) {</code>	Cria uma lista imutável de funcionários de entrada.
99	<code>return funcionarios.stream().map(this::calcular) .toList();</code>	Transforma cada funcionário em ResultadoFolha e coleta em lista.
100	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
101		Linha em branco para separar conceitos e melhorar legibilidade.
102	<code>ResultadoFolha calcular(Funcionario f) {</code>	Calcula a folha de um único funcionário.
103	<code>double extras = (f.salarioBruto() / 180.0) * 1.5 * f.horasExtras();</code>	Calcula horas extras: valor hora vezes 1,5 vezes quantidade.
104	<code>double inss = regraINSS.calcular(f.salarioBruto());</code>	Aplica a regra de INSS sobre o salário bruto.
105	<code>double baseIRRF = Math.max(0.0, f.salarioBruto() + extras - inss - f.dependentes() * DEDUCAO_POR_DEPENDENTE);</code>	Constante da dedução por dependente usada no IRRF.
106	<code>double irrf = regraIRRF.calcular(baseIRRF);</code>	Aplica a regra de IRRF sobre a base já ajustada.

Linha	Código	Explicação didática
107	<code>double liquido = f.salarioBruto() + extras - inss - irrf;</code>	Calcula salário líquido: bruto + extras - INSS - IRRF.
108	<code>return new ResultadoFolha(f.nome(), f.salarioBruto(), extras, inss, irrf, liquido);</code>	Cria o objeto de resultado com todos os valores finais.
109	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
110	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
111		Linha em branco para separar conceitos e melhorar legibilidade.
112	<code>record ResumoFolha(double totalFolha, ResultadoFolha maior, ResultadoFolha menor) {</code>	Record para total, maior e menor resultado da folha.
113	<code>static ResumoFolha de(List<ResultadoFolha> resultados) {</code>	Calcula todos os funcionários e obtém a lista de resultados.
114	<code>double total = resultados.stream().mapToDouble(Resultad oFolha::liquido).sum();</code>	Extrai os líquidos e soma para obter o total.
115	<code>ResultadoFolha maior = resultados.stream().max(Comparator.compa ringDouble(ResultadoFolha::liquido)).orE lseThrow();</code>	Falha se nenhuma faixa for encontrada, evitando resultado silenciosamente errado.
116	<code>ResultadoFolha menor = resultados.stream().min(Comparator.compa ringDouble(ResultadoFolha::liquido)).orE lseThrow();</code>	Falha se nenhuma faixa for encontrada, evitando resultado silenciosamente errado.
117	<code>return new ResumoFolha(total, maior, menor);</code>	Retorna o resumo consolidado.
118	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.
119	<code>}</code>	Fecha o bloco ou chamada iniciada anteriormente.

Resumo para apresentação oral

Explique que Java separa responsabilidades: records guardam dados, RegraDesconto define contrato, RegraPorFaixas reaproveita o algoritmo de faixas e CalculadoraFolha coordena o cálculo. É mais extensível, porém mais verboso.